금융 IT / 데이터 엔지니어 지원용 포트폴리오

김태형

TaeHyeong Kim

데이터가 빠지거나 틀리거나 두 번 처리되는 순간을 설계로 막아 왔습니다

경북대학교 컴퓨터학부 · 2027.02 졸업 예정 · xogud3624@gmail.com · github.com/KTHYEONG

증권사 API와 거래소 스트림에서 데이터를 모으고, 품질을 검증하고, 백업하고, 주문 직전까지 안전하게 연결하는 시스템을 혼자 만들고 운영했습니다. 장애가 났을 때 무엇을 복구했고 무엇을 못 했는지까지 측정하는 것을 기준으로 삼았습니다.

주요 역량

실시간 수집 파이프라인

증권사 웹소켓·REST를 비동기로 연동하고, 장중에는 파싱 없이 저장해 지연과 손실을 피했습니다.

데이터 정합성과 장애 복구

체결 시퀀스 감사, 보조 소스 교차 백필, 중복 제거 병합으로 누락 구간을 복구했습니다.

저장·백업 효율과 안전

압축 원본(zstd)에서 Parquet으로의 승격, 크기 대사 후 삭제 방식으로 작은 서버에서도 안전하게 순환했습니다.

운영 안정성

멱등 주문 저널, 원자적 파일 교체, 결정 시각을 피하는 배포 가드, AST 구조 검사로 실수가 사고가 되지 않게 했습니다.

대표 프로젝트

01 KRX-Alpha 개인

수집부터 검증·백업·주문 관리까지 데이터 인프라 전 구간

github.com/KTHYEONG/krx-alpha

02 K-Closing Alpha 개인

10분 안에 여러 외부 API에서 데이터를 모으고 주문까지 안전하게 연결하는 시스템

github.com/KTHYEONG/k-closing-alpha

03 SaveTax 개인

105만 건의 공공 데이터를 정제하고, 외부 웹 가격을 모아 근거와 함께 제출하는 파이프라인

비공개 저장소

04 Quant Dashboard 개인

운영 중인 수집·퀀트 파이프라인을 부하 없이 지켜보는 관제 도구

비공개 저장소

05 Port Entity Matching 개인

오타·표기 변형이 섞인 기록을 7만 8천 건 카탈로그에 연결하는 대규모 매칭

비공개 저장소

모든 수치에는 측정 방식(백테스트 · 재현 시뮬레이션 · 운영 기록 · README 기재)을 표시했고, 자세한 설명은 마지막 쪽에 있습니다.

PROJECT 01 개인 프로젝트

KRX-Alpha

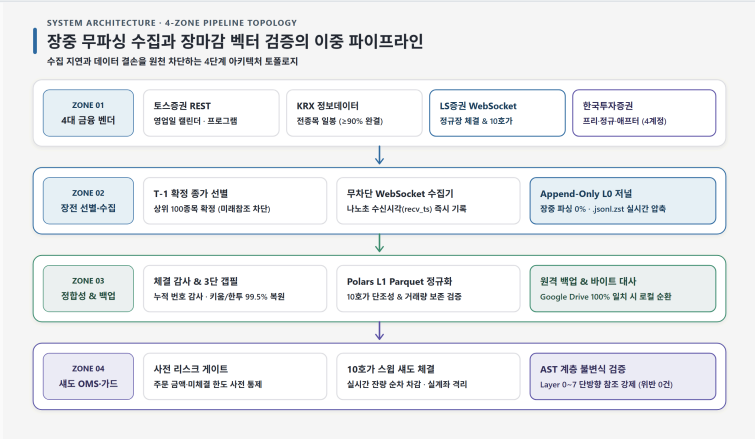
국내주식 체결·10호가 틱 데이터를 하루도 빠짐없이 모으고, 품질을 검증하고, 안전하게 백업하는 금융 데이터 파이프라인

역할	수집기·정규화·검증·백업·주문 관리(OMS) 계층까지 단독 설계·구현
기간	2026
기술	Python · asyncio/WebSocket · Polars · PyArrow(Parquet) · zstandard · SQLite · rclone · Docker Compose · Pydantic · pytest
GitHub	github.com/KTHYEONG/krx-alpha

이 직무 관점 **수집부터 검증·백업·주문 관리까지 데이터 인프라 전 구간.** 장중 무손실 수집, 장애 복구, 안전한 백업 순환, 모의·실전 분리를 다룹니다.

틱·호가 데이터는 장 시작 순간에 가장 많이 몰리고, 그때 한 번 놓치면 다시 얻기 어렵습니다. 이 프로젝트는 최대 90종목(프리·정규·애프터마켓)의 체결과 10단계 호가를 저사양 클라우드 서버에서 매일 무인으로 수집합니다.

장중에는 **파싱 없이 수신 시각만 붙여 압축 저장**해 지연과 손실을 피하고, 장 마감 후에 시퀀스 감사, 갭 복구, 10호가 단조성·거래량 보존 검증, Parquet 변환, 원격 백업 대사를 거쳐 로컬 원본을 순환 삭제합니다.



저장소 문서의 시스템 구조도. 4개 금융 벤더 → 장전 선별·무파싱 수집 → 정합성 검증·백업 → 섀도 OMS.

장중 틱 손실률
0.00%
수신 즉시 파싱 없이 append-only 저장
운영 기록

개장 직후 누락 복원
99.5%
15,873건 중 15,713건 · 2026-10-20 사고 1건 기준
운영 기록

백업 바이트 일치
247개 100%
삭제 전 원격 파일 크기 대조 · 디스크 여유 3.0GB 유지
운영 기록

구조 위반
0건
110개 모듈 · AST로 8개 계층의 단방향 참조 검사
README 기재

테스트
3,041개
품질 게이트(Ruff·Mypy·변경 영향 테스트) 포함
README 기재

핵심 판단과 해결

01 연결이 55초 끊겨 틱 15,873건이 사라졌을 때

문제

2026-10-20 09:00 개장 직후 증권사 서버 트래픽 폭주로 웹소켓이 55초간 끊겨, 42개 종목에서 체결 15,873건이 누락됐습니다. 수집 프로세스 자체는 정상(초당 1,586건 처리)이었습니다.

한 일

- 체결 틱의 고유 체결 카운터를 대조해 빈 구간의 시간대와 종목을 전수 산출
- 끊긴 증권사 대신 보조 증권사(키움·KIS)의 체결 조회 API로 해당 구간을 다시 채움
- SQLite 외부 정렬로 원본과 백필 데이터의 중복을 제거해 밀리초 단위로 병합

결과

15,713건(99.5%) 복원, 이후 정상 거래일의 최종 결손 0건(README 기재). 복원하지 못한 160건은 숨기지 않고 기록으로 남깁니다.

02 디스크가 가득 차기 전에 지우되, 백업이 안 된 건 절대 지우지 않기

문제

작은 VPS라 원본을 주기적으로 지워야 하는데, 단순 보존기간으로 지우면 네트워크 오류로 백업이 덜 된 데이터가 영구히 사라질 수 있었습니다. 반대로 백업 목록 전체를 매번 조회하면 수십 분이 걸려 마감 시간을 넘겼습니다.

한 일

- 날짜 접두사 단건 조회(rclone lsjson)로 원격 파일 크기만 확인하는 방식으로 대사 시간을 단축
- 메모리 부족으로 인한 변환 실패는 자동 재시도, 데이터 규격 오류는 격리 폴더로 분리해 파이프라인은 계속 진행
- 원격 크기가 로컬과 정확히 같고 복구 작업이 끝난 파일만 로컬에서 삭제(Offload-before-Delete)

결과

247개 파티션 바이트 일치 100%, 디스크 여유 최소 3.0GB를 사람 개입 없이 유지(README 기재).

03 모의 매매가 실전을 속이지 않게, 그리고 실계좌로는 실수로 나가지 않게

문제

모의 매매에서 "현재가에 전량 체결"로 가정하면 실전의 호가 잔량 부족과 슬리피지를 알 수 없습니다. 반대로 같은 코드가 실계좌 주문까지 낼 수 있으면 사고 위험이 있습니다.

한 일

- 가상 주문도 실시간 10호가의 실제 잔량을 순서대로 깎아 체결가와 수량 산출
- 하나의 OMS에서 주문 전문을 실전과 똑같이 만들되, LIVE_ARMED=true가 명시되지 않으면 소켓 송출만 물리적으로 차단
- Python AST로 8개 계층의 단방향 참조와 환경변수 직접 접근 금지를 CI에서 검사. 장중 배포는 자동 동결

결과

모의와 실전이 같은 리스크 게이트를 통과하고, 110개 모듈 구조 위반 0건을 유지합니다.

한계

- 이 프로젝트는 알파가 아니라 데이터 인프라입니다. 수집한 데이터로 만든 수익 성과는 없습니다.
- 복원률은 2026-10-20 사고 한 건의 수치입니다. 다른 유형의 장애 빈도는 아직 충분히 쌓이지 않았습니다.
- 실계좌 주문 기능은 기본적으로 차단된 상태로 운영하며, 실전 체결 성과는 없습니다.

배운 점

장애는 반드시 나므로, 무엇을 얼마나 못 복구했는지까지 측정할 수 있게 만드는 것이 신뢰성의 시작이라는 점을 배웠습니다.

PROJECT 02 개인 프로젝트

K-Closing Alpha

15:20에 알 수 있는 정보만으로 종목을 고르고, 종가에 사서 다음 날 시초가에 파는 국내주식 퀀트 시스템

역할	기획·데이터 수집·피처·모델·검증·주문 실행 로직까지 전 과정을 단독 설계·구현
기간	2026
기술	Python · LightGBM · Pandas · PyArrow(Parquet) · aiohttp · Pydantic · pytest · systemd
GitHub	github.com/KTHYEONG/k-closing-alpha

이 직무 관점 **10분 안에 여러 외부 API에서 데이터를 모으고 주문까지 안전하게 연결하는 시스템.** 증권사별 호출 제한과 토큰 제한 아래에서의 수집 설계와, 타임아웃에도 중복 주문이 나가지 않는 주문 저널이 중심입니다.

한국 시장의 장 마감 동시호가(15:20~15:30)는 기관·외국인의 포지션 정리와 패시브 수급이 몰려 하루 중 수급 불균형이 가장 큰 구간입니다. 그러나 이 구간을 백테스트하면 마감 후 확정되는 종가가 판단에 섞이기 쉽고, 저가주는 호가 비용과 거래세가 수익을 지우며, 10분 안에 데이터를 모아야 하는 현실적 제약도 있습니다.

결정창 수집
95초 이내
최대 1,247개 후보 단면 · 쿼터 차단 0건
운영 기록

중복 주문
0건
Write-Ahead 저널 · 모의투자 원장 기준
운영 기록

검증 기간
886거래일
2023-01 ~ 2026-09 · 68,493개 일별 종목 단면
백테스트

핵심 판단과 해결

01 10분 결정창 안에 전 종목 데이터 모으기

문제

15:20~15:30 사이에 전 종목 스캔, 10호가, 예상체결, 투자자 수급을 모아야 하는데 증권사 한 곳의 호출 제한(초당 18건 등)으로는 시간 안에 끝낼 수 없었고, 여러 프로세스가 동시에 뜨면 토큰 발급 제한까지 걸렸습니다.

한 일

- 증권사별 강점에 맞춰 역할 분담. 키움은 전 종목 스캔(1회 200종목), KIS는 10호가·수급, DB금융투자·LS는 분봉, 토스는 시세 교차 확인
- 여러 프로세스가 토큰과 호출 간격을 하나의 파일 락(flock)과 세대 번호로 공유하도록 하고, 매일 07:05에 토큰을 미리 발급
- KIS 키를 용도별 슬롯으로 나눠, 15:20 실시간 경로가 장후 백필 작업의 호출 한도에 영향받지 않게 격리

결과

README 기재 기준 최대 1,247개 후보를 95초 안에 수집하고 호출 제한·토큰 충돌 차단 0건.

02 타임아웃 때 "다시 보내지 않기", 종가가 확정되기 전엔 "기록하지 않기"

문제

주문 응답이 타임아웃되면 접수 여부를 알 수 없고, 그대로 재전송하면 이중 주문이 됩니다. 또 15:30 직후 일부 시세 API는 아직 확정되지 않은 예상체결가를 돌려주어 종가 장부가 틀어질 수 있었습니다.

한 일

- 전송 직전 주문 의도를 디스크에 fsync로 먼저 기록(Write-Ahead). 타임아웃 시 상태를 UNKNOWN으로 두고 브로커 원장을 조회해 이미 접수됐으면 채택, 아니면 종료
- 종가는 ① 15:30 도달 ② 장운영 구분코드 '3' ③ 현재가·호가 TR의 종가 일치, 세 조건이 모두 맞을 때만 확정하고 아니면 백오프 재조회 후 미확정으로 격리

결과

README 기재 기준 모의투자 원장에서 중복 주문 0건, 현금·포지션·NAV 정산 오차 0원.

한계

- 118개 전략을 탐색한 뒤 고른 결과라 선택 편향이 남아 있습니다. CPCV와 엠바고로 줄였지만 없애지는 못했습니다.
- 표본이 약 3년 8개월의 한 시장입니다. 다른 국면에서의 재현은 확인하지 못했습니다.

PROJECT 03 개인 프로젝트

SaveTax

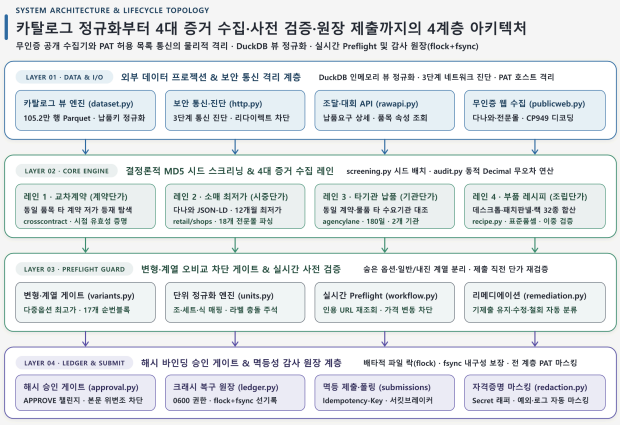
105만 건의 공공조달 데이터에서 세금이 과다 집행된 품목을 찾아, 근거와 함께 제보하는 감사 엔진

역할	데이터 정규화, 4가지 시장가 증거 수집, 오비교 방지 규칙, 중복 없는 제출 원장까지 단독 설계·구현 (SaveTax 해커톤 참가)
기간	2026.1
기술	Python · DuckDB · Parquet · Decimal · pytest · mypy(strict) · ruff
GitHub	비공개 저장소

이 직무 관점 **105만 건의 공공 데이터를 정제하고, 외부 웹 가격을 모아 근거와 함께 제출하는 파이프라인.** 중복으로 부풀려진 집계를 바로잡는 데이터 정제, 쇼핑몰마다 다른 형식을 다루는 수집, 네트워크 오류에도 중복 없는 제출이 중심입니다.

SaveTax 해커톤은 과도하게 집행된 세금의 흔적을 근거와 함께 찾아 제보하는 대회입니다. 나라장터 납품 카탈로그는 규모가 크고 중복이 많아, 숫자를 그대로 믿으면 오히려 틀린 제보를 만들게 됩니다.

그래서 데이터 정규화로 모수를 바로잡고, 시중가·타 계약·타 기관·조립 원가의 네 경로로 가격 근거를 모은 뒤, 오비교를 막는 규칙과 제출 직전 재확인을 거쳐 제보하는 파이프라인을 만들었습니다. 저장소는 비공개입니다.



저장소 문서의 구조도. 데이터 추출·보안 통신 → 증거 수집 → 오비교 방지 게이트 → 승인·중복 방지 제출.

분석한 조달 데이터

105.2만 건

47MB Parquet를 DuckDB로 메모리 내 정규화 · 고유 납품번호 300,431개

README 기재

바로잡은 집계 왜곡

4.19조 원

중복 매핑으로 17.57조 원이 21.76조 원 (+23.9%)으로 부풀려진 것을 제거

README 기재

해커톤 제출 결과

10.2억 원

제보 슬롯 100/100 사용 · 유효 수락 78건 · 수락 품목 기준 10억 2,142만 9,243원

운영 기록

중복 제보

0건

네트워크 오류 후 재실행에서도 슬롯 중복 차감 없음

운영 기록

핵심 판단과 해결

01 부풀려진 모수를 먼저 바로잡고, 비교 대상을 잘못 고르지 않기

문제

카탈로그 한 행이 아니라 납품번호(dlv_key) 하나가 여러 등재 품목에 중복 매핑되어, 그대로 합치면 전체 주문액이 17.57조 원에서 21.76조 원으로 23.9% 부풀려졌습니다. 또 같은 물품번호 안의 저가 옵션이나 일반형 모델을 고가 납품건과 비교하면 허위 제보가 됩니다.

한 일

- 고유 납품번호별 계약번호·금액 무결성을 전수 검사해 중복을 제거한 정규화 뷰 구축(DuckDB)
- 단가가 여러 개면 가장 높은 대표 단가만 비교 기준으로 쓰고, 일반형·내진형처럼 순번 블록이 다른 모델은 교차 비교하지 않도록 계열 분리
- 비교하는 계약이 해당 납품일에 실제로 유효했는지 변경계약일과 납품 이력으로 확인

결과

300,431개 고유 납품번호 기준으로 모수를 산출했고 과대 계상을 제거했습니다. 잘못된 비교로 나가는 제보를 규칙으로 막았습니다.

02 시중에 같은 모델이 없는 조달 전용 품목의 기준가 만들기

문제

가장 큰 금액의 데스크톱 PC, 패치판넬, 서버랙은 공공기관 전용 모델명이라 쇼핑몰에서 같은 제품을 찾을 수 없었습니다. 쇼핑몰마다 문자 인코딩(CP949) 깨짐, 숨은 가격 태그, 배송 문구와 품절 표기가 섞여 자동 판정이 흔들렸습니다.

한 일

- 품목 사양(CPU·RAM·SSD·규격)을 정규화해 동급 시판 부품으로 분해하고, 2026 표준품셈의 설치 공임 5종을 더한 조립 원가 모델 구축
- 한글 인코딩 자동 복구, 숨은 태그 선제거, 품절 문구 정규식 필터로 쇼핑몰 파서 보강
- 부품마다 서로 다른 2개 쇼핑몰 가격을 수집하고, 기준가 대비 1.2~1.3배 경계에 걸리면 2순위 가격에서도 1.2배 이상일 때만 통과

결과

32종 표준 부품과 5종 공임을 자동 수집해 조달 전용 모델에서 적발 금액의 대부분(데스크톱 PC 6.6억 원, 패치판넬 1.9억 원, 서버랙 1.7억 원)을 찾았습니다.

03 100번뿐인 제보 기회를 네트워크 오류에도 잃지 않기

문제

제보는 참가자당 100회로 제한되어, 타임아웃이나 중단 후 재실행에서 같은 품목이 중복 차감될 위험이 있었습니다. 인증키(PAT)가 로그나 리다이렉트로 새는 위험도 있었습니다.

한 일

- 전송 전에 파일 락(flock)과 fsync를 쓴 로컬 원장에 고유 요청 키를 먼저 기록하고, 재실행 시 미결 요청을 자동 복구
- 제출 직전에 쇼핑몰 URL을 다시 조회해 품절·가격 변동이 있으면 제보를 자동 취소
- 공식 호스트 3곳 외로는 전송을 차단하고 리다이렉트를 막으며, 모든 로그에서 키를 자동 마스킹

결과

중복 청구 0건으로 100개 슬롯을 모두 사용했고, 키 유출은 없었습니다 (README 기재).

한계

- 10.2억 원은 해커톤 시스템의 수락 기준 금액이며, 실제 환수나 과다 집행 확정을 뜻하지 않습니다. 대회 순위와 수상은 포함하지 않았습니다.
- 적발의 약 65%가 데스크톱 PC 한 범주에 집중되어, 품목 구성이 달라지면 같은 방식의 성과를 기대하기 어렵습니다.
- 조립 원가 모델은 표준품셈과 판매처 가격에 의존하므로, 가격 변동이나 사양 정규화 오류가 기준가를 흔들 수 있습니다.

배운 점

큰 숫자를 찾는 것보다, 그 숫자가 잘못된 비교에서 나온 것이 아님을 규칙으로 증명하는 일이 더 많은 시간을 차지한다는 것을 배웠습니다.

PROJECT 04 개인 프로젝트

Quant Dashboard

매일 도는 퀀트·수집 파이프라인이 정상인지, 트레이딩 프로세스를 방해하지 않고 지켜보는 읽기 전용 관제 도구

역할	상태 수집기, 시장 캘린더 판정, 이벤트·리포트 저장, 배포 구성까지 단독 설계·구현
기간	2026.1
기술	Python(표준 라이브러리만) · systemd · Tailscale · 순수 HTML/CSS/JS · pytest · mypy(strict)
GitHub	비공개 저장소

이 직무 관점 **운영 중인 수집·퀀트 파이프라인을 부하 없이 지켜보는 관제 도구.** 정상 상태를 장애로 오판하지 않는 규칙과, 대상 시스템에 I/O 부담을 주지 않는 수집 방식을 보여 줍니다.

런타임 외부 의존성
0개
표준 라이브러리만 사용
README 기재

수집기 자원 상한
128MB · CPU 25%
systemd cgroup으로 본 트레이딩 프로세스와 분리
README 기재

틱 파일 읽기
0 Byte
대용량 압축 파일은 열지 않고 수정 시각(mtime)만 확인
README 기재

핵심 판단과 해결

01 정상인데 장애로 보이는 상태를 구분하기

문제

휴장일·주말에 시세 수집기가 켜지지 않는 것은 정상인데, 하트비트가 없다고 FAIL로 판정해 불필요한 알람이 나가고 가용성 지표도 왜곡됐습니다. 15:20~15:30 동시호가에는 체결 틱이 없는 것이 정상이라, 120초 지연 규칙이 매일 같은 시각에 오탐했습니다.

한 일

- 거래소 연간 일정을 반영한 캘린더로 당일을 5가지(거래일·휴장일·주말·이동·미확인)로 판정
- 휴장일에 멈춰 있는 타이머는 결측이 아니라 CLOSED_DAY_NOOP로 분리하고, 동시호가 구간은 체결 판정을 보류했다가 15:40 이후 애프터마켓 규칙으로 인계
- 정규장·동시호가·애프터마켓 시간대별 판정을 단위 테스트로 고정

결과

설계상 휴장일·동시호가 허위 경보를 제거했습니다(README 기재).

한계

- 기능은 관제와 리포트에 한정되며, 장애 자동 복구는 하지 않습니다.
- 허위 경보 제거 효과는 회귀 테스트와 설계 기준이며, 장기 운영 통계로 측정한 값은 아닙니다.
- 저장소는 비공개이며 개인 사설망 안에서만 운영합니다.

PROJECT 05 개인 프로젝트

Port Entity Matching

오타와 누락이 섞인 입출항 기록을 7만 8천 척의 표준 선박 카탈로그에 연결하는 매칭 모델 (DACON 항만 AI 대회)

역할 정규화, 후보 검색, 학습 랭커, 평가 도구, LLM 음역 채널까지 대회 개인 참가로 단독 설계·구현

기간 2026.1

기술 Python · LightGBM(lambdarank) · LLM(Kanana·Qwen · 고정 가중치) · pytest · uv

GitHub 비공개 저장소

이 직무 관점 **오타·표기 변형이 섞인 기록을 7만 8천 건 카탈로그에 연결하는 대규모 매칭.** 표기 정규화와 인덱스 기반 후보 생성으로 전수 비교를 피하고, 한 행의 결과가 다른 행에 의존하지 않도록 설계했습니다.

데이터 규모
78,595척
카탈로그 · 학습 12,222행
README 기재

후보 생성 속도
90초
학습 12,222행 전체 · 인덱스 구축 1.5초 · 최대 메모리 0.25GB
README 기재

로컬 교차검증 점수
0.9875
선박 단위로 분리한 5-fold · 한글명 0.9824 · 비한글명 0.9995 · 가중 0.7/0.3
백테스트

핵심 판단과 해결

01 표기 잡음을 규칙과 인덱스로 풀기

문제

한글 선박명은 영문 카탈로그명의 한글 표기라 외래어(리플렉션), 로마자(흥아), 약어(케이엠티씨), 번호(제2)가 뒤섞이고, 영문명은 1~2글자 오타가 있고, 톤수도 ±3~5% 변형돼 있습니다. 한글 행은 데이터의 9.7%인데 점수의 70%를 차지합니다.

한 일

- 유니코드 정규화와 기호 정리, 한글 자모 기반 음운 키, 톤수 허용 구간을 결합한 후보 검색. 7만 8천 행 전체를 쌍으로 비교하지 않고 인덱스로 후보를 좁힘
- 영문 문자열 유사도, 한글 음운 유사도, 톤수 일치, 숫자 일치 등 쌍 특성 40여 개로 LightGBM 랭커 학습. 선박 단위로 분리한 교차검증으로 평가
- 다른 테스트 행의 통계를 쓰지 않는 행 독립 추론 계약을 코드와 테스트로 강제

결과

선박 분리 5-fold 교차검증 점수 0.9875(한글명 0.9824). 후보 생성 재현율 1.000은 일부 기억을 포함한 진단이라 낙관적인 값으로 따로 표기합니다.

한계

- 대회가 2026-11-02까지 진행 중이라 최종 순위와 비공개 리더보드 점수는 아직 없습니다.
- 교차검증(0.9875)과 리더보드(0.9163)의 격차는 아직 남아 있습니다. 한글 규칙을 학습 데이터 이름을 본 뒤에 작성해 로컬 점수가 낙관적일 수 있음을 문서에도 적었습니다.
- 서버 시간 예산은 L4 서버 실측이 아니라 로컬 GPU 기반 추정치이며, 대회 규칙상 데이터와 코드는 외부에 공유할 수 없어 저장소는 비공개입니다.

> **수치를 읽는 법**
>
> 모든 수치에는 측정 방식을 함께 표시했습니다. 「백테스트」는 과거 데이터 시뮬레이션, 「재현 시뮬레이션」은 원장 리플레이, 「README 기재」는 저장소 문서에 적힌 운영 기록입니다. 실계좌 수익이나 수상 실적은 이 자료에 포함하지 않았습니다.
>
> 백테스트 과거 데이터 백테스트 재현 시뮬레이션 체결·증거금 규칙을 적용한 원장 리플레이 운영 기록 저장소 README에 기재된 운영 기록 README 기재 저장소 문서에 기재된 값

그 밖의 프로젝트와 링크

K-Stock Engine	국내주식 실행 코어와 KIS·KRX·DART 연동을 분리한 모듈형 엔진. 과거 연구 코드는 legacy/로 격리했습니다.
OIBC 2026	태양광 발전량 예측과 포트폴리오 최적화 챌린지 프로젝트. 진행 중이라 검증된 성과는 싣지 않았습니다.
ETF-Manager	월 적립식 ETF 자산배분 연구(S&T / 퀀트 자료에 요약 수록).

연락 xogud3624@gmail.com
GitHub https://github.com/KTHYEONG
금융·퀀트 Notion https://sparkly-quince-4ff.notion.site/Portfolio-3c996e29c4cf80c38b09ea1cca9a1cb4
AI Notion https://sparkly-quince-4ff.notion.site/AI-Portfolio-3e396e29c4cf810aaa44df3204963b41